

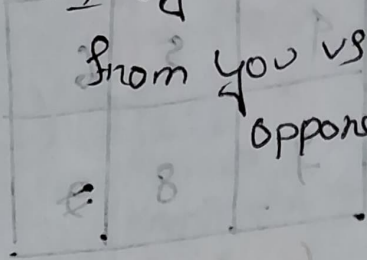
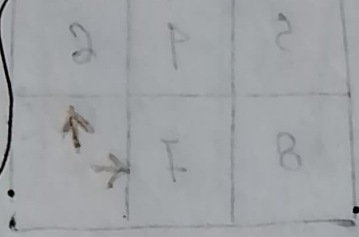
TIC-TAC-TOE

(2 player game)

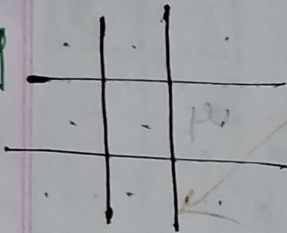
BFS
DFS
A*
Greedy

Player control
21000

but say for tic-tac-toe
1 by 1 attempt
from you vs
opponent



Starting
State
(9)



Starting 0 on X place 0011 possible
states 96

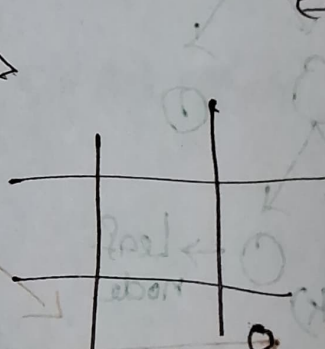
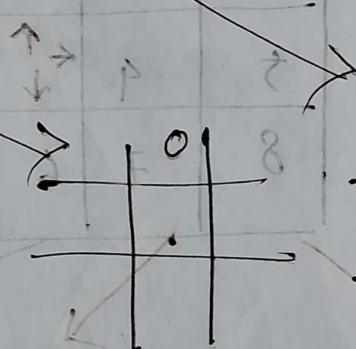
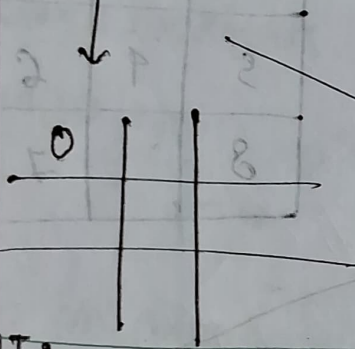
MY TURN:

winning max

9 possible turns

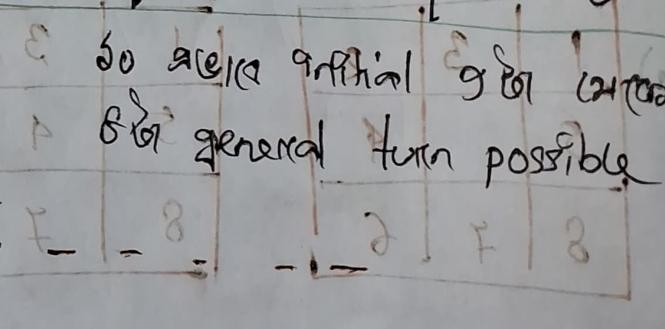
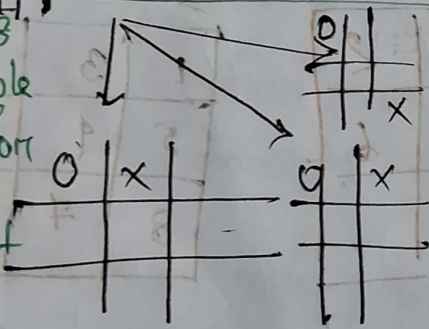
6 more states
(6)

For this
state
empty
place 8



OPPONENT

now 8
possible
turns for
the
opponent

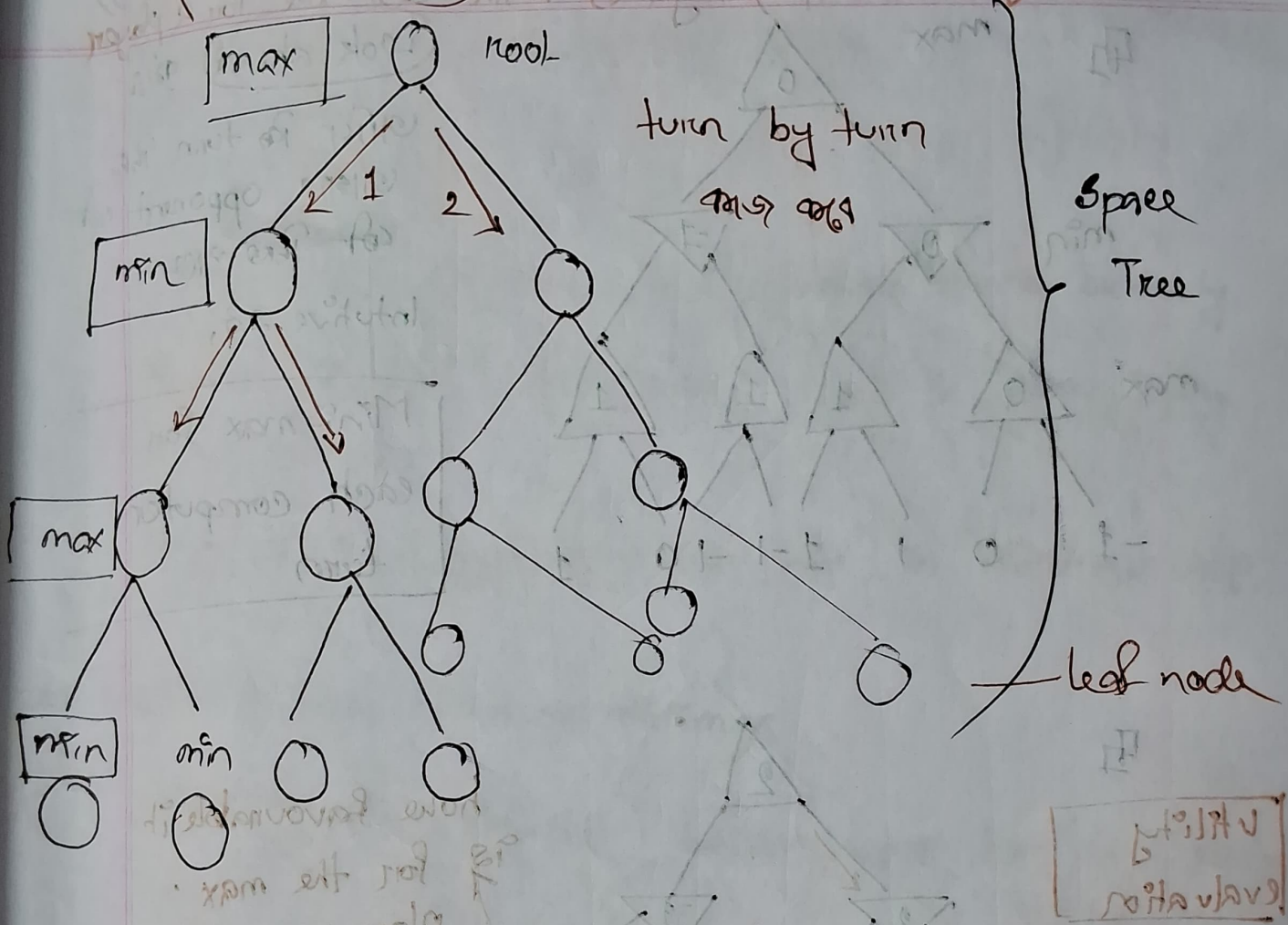


So after initial 9 for 21000
8 for general turn possible

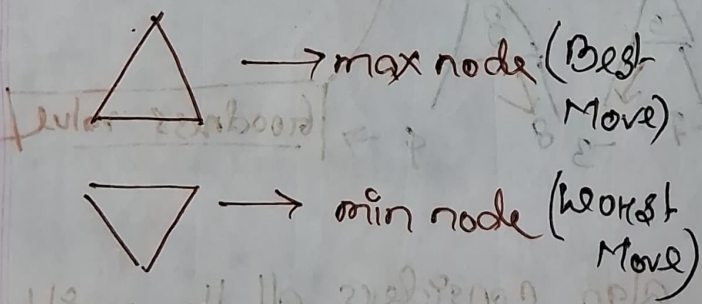
In my turn, I try to maximize my
winning possibility

In opponent's turn, he tries to maximize
his winning possibility / minimize my winning possibility

* minimum-maximum alternates (2 player game)



So, this is how mini-max algorithm works



Goal state

win = 1

lose = -1

draw = 0

* Max will try to maximize utility

* Min " " minimize " "

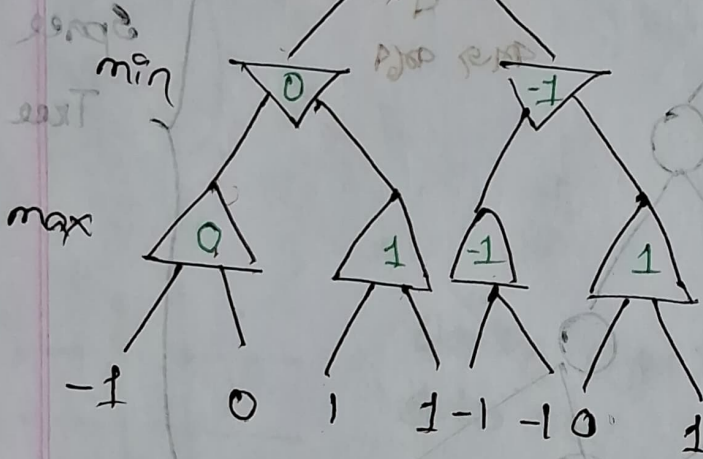
Mini-max $\rightarrow O(b^d)$ $b \rightarrow$ branching factor $d \rightarrow$ depth (ply)

The algo in two player games to find best move for a player

□

max

Look ahead

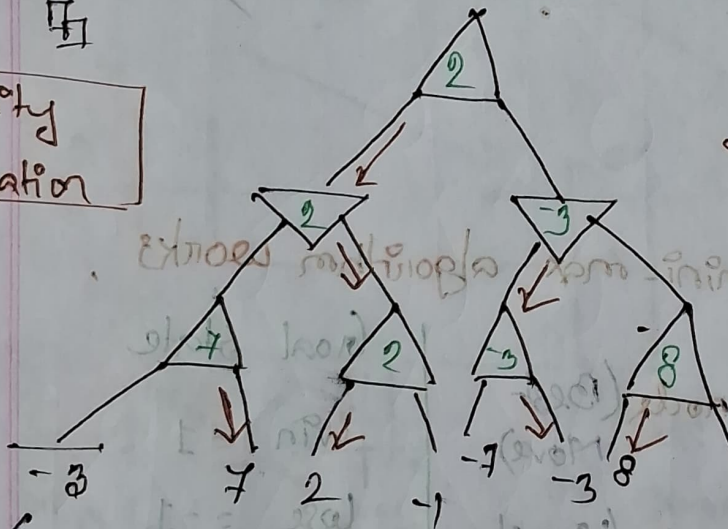


બીજા પાસે ફરતે પાસે
બીજા opponent નો
જો શ્રેષ્ઠ માર્ગ
Intuitive રીતે,

Mini-max for
each computer
turn

□

Utility
evaluation



how favourable it
is for the max
player

Goodness value

બીજા પાસે
બીજા;
બીજા પાસે
બીજા opponent
જો શ્રેષ્ઠ માર્ગ
(રોક)

The algo considers all the possible
moves, plays them out until a predefined
depth or till the end of the game, and
then evaluate the board pos.

Evaluation func: To determine the goodness value of a game state, you'll need an evaluation func. This func to calculates the score of a given state.

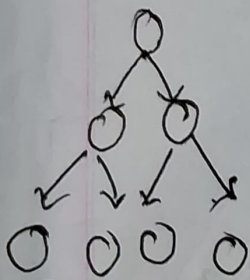
⇒ games-like dice, cards, pubg, valorant cannot be solely be part of this algo, because luck is also playing a major role

only rational thing(logic) can help in mini-max algo.

zero-sum, tic-tac-toe, nimb, ⇒ minimax

chess ⇒ Not minimax because space tree will be vast

$O(b^d)$



in mini max, we explore every node.

Alpha-Beta pruning $\rightarrow O(b^{d/2})$ best case
 $\rightarrow O(b^d)$ worst case
 PRUN $\rightarrow$ CUT OFF

* Cut off the search by exploring less no. of nodes
 * In alpha-beta pruning, we cut off ^{search} exploring every node.

$\Rightarrow$ time complexity gets reduced. Efficient than minimax

$\alpha \rightarrow$ max node

$\beta \rightarrow$ min node

max

min

max

min

terminal

